# Браузерное окружение

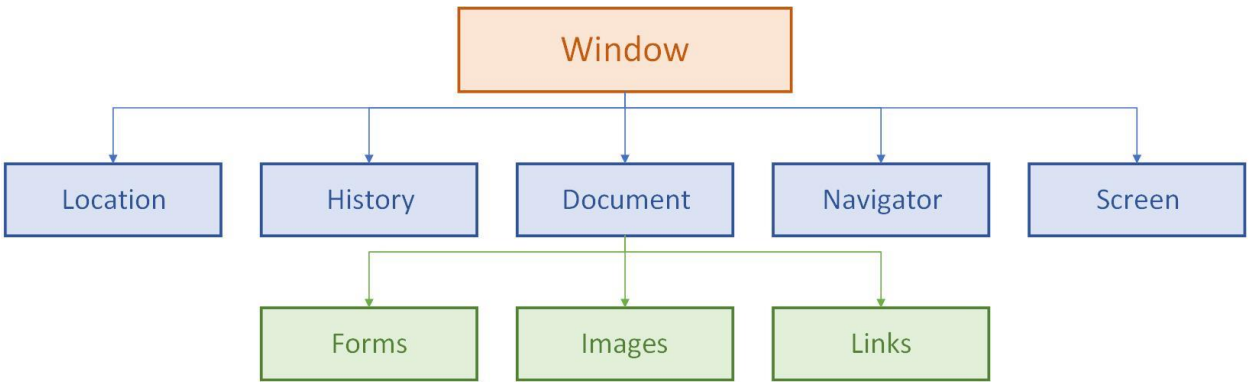
- Введение в Browser Object Model (BOM)
- Объект window
- Объект screen
- Диалоговые окна window
- Поиск на странице

## Введение в Browser Object Model (BOM)

Большое значение в JavaScript имеет работа с веб-браузером и теми объектами, которые он предоставляет. Например, использование объектов браузера позволяет манипулировать элементами HTML, которые имеются на странице, или взаимодействовать с пользователем.

Все объекты, через которые JavaScript взаимодействует с браузером, описываются таким понятием как **Browser Object Model** (Объектная Модель Браузера).

Browser Object Model (BOM) в упрощенном виде можно представить в виде следующей схемы:




В вершине находится главный объект - объект **window**, который является глобальным объектом (фактически представляет собой **браузер**). Этот объект в



свою очередь включает ряд других объектов, в частности, объект **document**, который представляет отдельную **веб-страницу**, отображаемую в браузере.

# Объект window

Объект **window** представляет собой окно веб-браузера, в котором размещаются веб-страницы. Window является **глобальным объектом**, поэтому при доступе к его свойствам и методам необязательно использовать его имя.

Например:

Объект **window** имеет метод **alert**(), который отображает окно сообщения. Но нам необязательно писать:

```
window.alert("Привет мир!");
```

можно использовать **сокращенную** форму записи:

```
alert("Привет мир!");
```

Глобальность, накладывает некоторые **особенности** на работу с этим объектом.

Если объявить в программе какую-нибудь глобальную переменную, то она станет доступна как **новое свойство объекта window**:

```
var message = "Hello World";
console.log(window.message);
```

Если имя объявленной переменной совпадает с именем стандартного свойства, **стандартное свойство window будет переписано**.

> ***Примечание***. Такое поведение касается не только свойств. Переопределить можно как стандартное свойство, так и стандартный метод (об этом далее).



В демонстрационном примере **example_1** с помощью **var** определяется **глобальная переменная alert**. Все объявляемые в программе глобальные переменные или функции **автоматически добавляются к объекту window**.

Поскольку название новой переменной будет совпадать с названием стандартного метода **alert**(), то произойдет переопределение этого метода в объекте **window** новой переменной.

**example_1**.

```
<script>
    // alert - стандартное диалоговое окно
    window.alert("Привет мир!");
    var alert = "Что у вас здесь происходит?";
    // а теперь строковая переменная
    document.write('<h1>' + window.alert + '</h1>');
</script>
```

Эта особенность может иметь как положительный **эффект**, так и отрицательный. Рассмотрим код демонстрационного примера **example_2**.

**example_2**. **index.html**

```
<script>
    // простая строковая переменная
    message = "Hello World";
    console.log(window.message);
    // создаем свойства на глобальном window
    var name = 'Евгений';
    var login = 'master';
    var pwd = 'qwerty@';
    console.log(name);
```

```
    console.log(login);
    console.log(pwd);
</script>
<!-- перейдем по ссылке -->
<a href='target.html'>Go forward!</a>
```

Глобальное свойство **name** переписано пользователем, такое поведение свойства может стать неожиданностью в других скриптах.

**example_2. target.html**

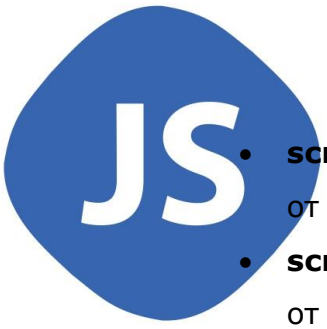
```
<script>
    // свойство name переписывает стандартное свойство window
    document.write ("<h1>Здравствуйте, " + window.name +
    '!</h1>');
</script>
```

Так что эффект скорее все-таки отрицательный ...

## Свойства объекта window

С помощью свойств объекта **window** можно получить различную информацию об **окне браузера**. В частности, для определения позиционирования окна применяются следующие свойства:

- **innerHeight –** содержит высоту окна, в том числе горизонтальные полосы прокрутки;
- **innerWidth –** содержит ширину окна, в том числе вертикальные полосы прокрутки;
- **outerHeight** – содержит высоту окна браузера, в том числе все полосы прокрутки браузера;
- **outerWidth** – содержит ширину окна браузера, в том числе все полосы прокрутки браузера;



- **screenX** – содержит позицию окна браузера по оси X, то есть расстояние от окна браузера до левого края экрана;
- **screenY** – содержит позицию окна браузера по оси X, то есть расстояние от окна браузера до верхнего края экрана;
- **scrollX** – содержит количество пикселей веб-страницы, прокрученных по горизонтали;
- **scrollY** – содержит количество пикселей веб-страницы, прокрученных по вертикали

Получим информацию об окне **браузера** (объекте **window**).

**example_3**.

```
<script>
    // тестируем свойства объекта window
    // высота окна
    console.log('innerHeight: ' + window.innerHeight);
    console.log('outerHeight: ' + window.outerHeight);
    // ширина окна
    console.log('innerWidth: ' + window.innerWidth);
    console.log('outerWidth: ' + window.outerWidth);
    // позиция окна
    console.log('screenX: ' + window.screenX);
    console.log('screenY: ' + window.screenY);
</script>
```

***Примечание***. Не стоит пытаться запомнить все имеющиеся свойства и методы. Самые главные запомнятся автоматически, а не самые главные – посмотрим во всякого рода справочниках.



А вот что самое главное, а что не самое, зависит от вашего рода деятельности как frontend разработчика.

## Методы объекта window

Помимо свойств объект **window** обладает целым набором разных методов. Вот только **некоторые** из них.

- **alert**() – отображает окно предупреждения с сообщением и кнопкой ОК
- **blur**() – удаляет фокус из текущего окна
- **clearInterval**() – очищает таймер , установленный с setInterval()
- **clearTimeout**() – очищает таймер , установленный с setTimeout()
- **close**() – закрывает текущее окно
- **confirm**() – отображает диалоговое окно с сообщением и ОК и кнопку Отмена
- **focus**() – устанавливает фокус на текущее окно
- **moveBy**() – перемещение окна относительно его текущего положения
- **moveTo**() – перемещение окна в заданное положение
- **open**() – открывает новое окно браузера
- **print**() – печатает содержимое текущего окна
- **prompt**() – отображает диалоговое окно с указанным текстом и полем для пользовательского ввода
- **resizeBy**() – изменение размера окна на указанное количество пикселей относительно его текущего размера.
- **resizeTo**() – изменение размера окна до заданной ширины и высоты
- **scrollBy**() – прокрутка документ на указанное число пикселей
- **scrollTo**() – прокрутка документа для заданных координат
- **setInterval**() – вызывает функцию или вычисляет выражение через заданные интервалы времени
- **setTimeout**() – вызывает функцию или вычисляет выражение после определенного количества миллисекунд
- **stop**() – остановки окна от загрузки

***Примечание***. Наиболее востребованные методы рассмотрим дальше отдельным разделом (или даже отдельным уроком).

**example_4**.

```
<div>
    <h2>«Дело не в том, что у нас мало времени, а в том, что мы тратим его впустую». — Сенека</h2>
    ... и тем не менее: <b><span id="timer"></span></b>
</div>
<script>
    let timer = document.getElementById("timer");
    let i=0;
    const intervalId = setInterval(function() {
        timer.innerText = i;
        if (i==10) {
            if(confirm('Прекратить тратить время впустую?'))
            clearInterval(intervalId);
        }
        console.log('Время утекает каждую секунду -> ' + i++)
    }, 1000)
</script>
```

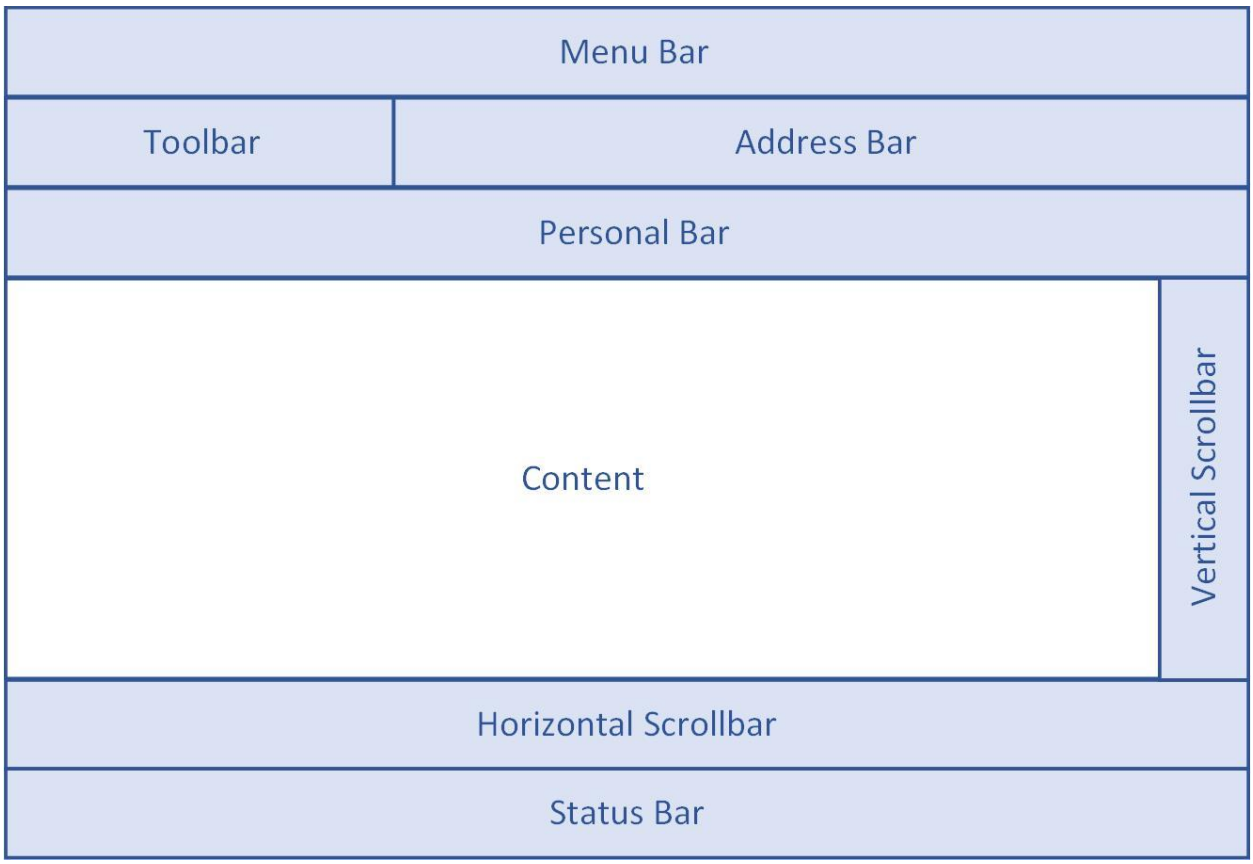
***Примечание***. Методы **setInterVal**() и **setTimeout**() будем широко использовать в асинхронной разработке.

## Компоненты браузера

Окно браузера обычно состоит из различных компонентов. В зависимости от конкретного браузера и персональных настроек набор конкретных компонентов может отличаться. Но в общем случае типовое окно браузера выглядит следующим образом:

Помимо собственно области содержимого, в которой отображается веб-страница, веб-браузер имеет имеет еще ряд компонентов:

- **адресная строка** для ввода URL-адреса или поискового запроса (Address Bar);
- **строка состояния** (Status Bar), которая указывает, загружена ли веб-страница или находится в процессе загрузки;
- **строка меню** (Menu Bar);
- **панель инструментов** (Tool Bar);
- **персональная** панель, которая, например, содержит закладки (Personal Bar);
- **полосы прокрутки**, которые отображают горизонтальное и вертикальное положение (Horizontal/Vertical ScrollBar).



Объект **window имеет ограниченные возможности для взаимодействия с этими компонентами**, в частности, для **проверки наличия** этих компонентов объект window имеет ряд свойств:

- **locationbar** – содержит объект, который указывает, отображается адресная строка или нет;
- **menubar** – указывает, отображается ли панель меню или нет;
- **personalbar** – указывает, отображается ли персональная панель (например, панель закладок) или нет;
- **scrollbars** – указывает, отображаются ли полосы прокрутки или нет;
- **statusbar** – указывает, отображается строка состояния или нет;
- **toolbar** – указывает, отображается ли панель инструментов или нет.

Например, узнаем отображаются ли некоторые из панелей. Каждое свойство возвратит **объект BarProp**, в свою очередь объект имеет свойство **visible**, которое укажет, отображается панель или нет.

**example_5**.

```
<script>

    // тестируем компоненты браузера

    console.log('Adress Bar: ' + window.locationbar + ' -> ' +
    window.locationbar.visible);

    console.log('Menu Bar: ' + window.menubar + ' -> ' +
    window.menubar.visible);

    console.log('Status Bar: ' + window.statusbar + ' -> ' +
    window.statusbar.visible);

    console.log('Personal Bar: ' + window.personalbar + ' -> ' +
    window.personalbar.visible);

    console.log('Scroll Bars: ' + window.scrollbars + ' -> ' +
    window.scrollbars.visible);

</script>
```



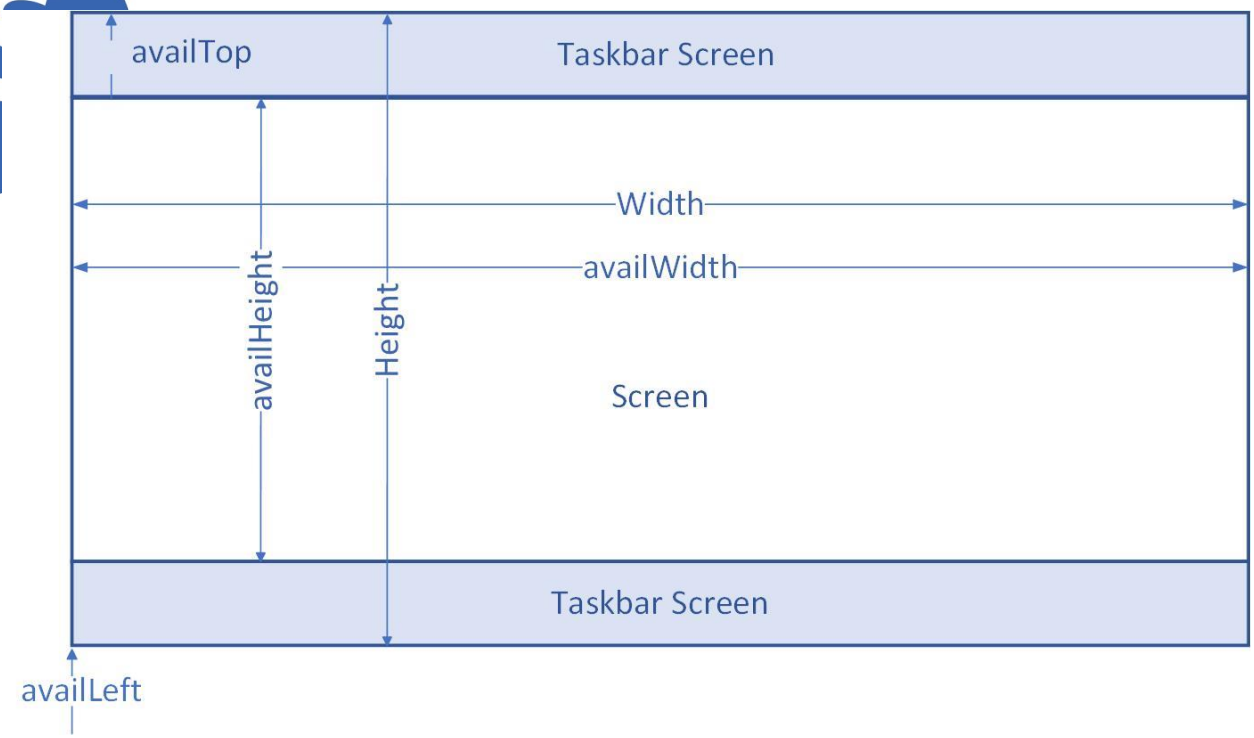
# Объект screen

Для получения информации об экране также применяется свойство **screen** объекта **window**. Это свойство представляет объект типа **screen**.

> ***Примечание***. **Свойство screen** представляет из себя **объект screen**. Именно поэтому текущий раздел с одинаковым успехом можно назвать:
>
> - **Свойство screen**
> - **Объект *s*creen**

Свойства объекта типа **screen**:

- **availTop** – указывает на высоту фиксированных компонентов, которые примыкают к верхней стороне браузера, например, различных верхних панелей;
- **availLeft** – указывает на ширину фиксированных компонентов, которые примыкают к левой стороне браузера, например, различных левых панелей;
- **availHeight** – содержит максимально доступную высоту в пикселях минус высоту верхних и нижних панелей (сравните с height);
- **availWidth** – содержит максимально доступную ширину в пикселях минус ширину левых и правых панелей (сравните с width);
- **colorDepth** – содержит глубину цвета экрана;
- **height** – содержит высоту экрана в пикселях (сравните с availHeight);
- **orientation** – содержит **объект** типа **ScreenOrientation**, который предоставляет информацию об ориентации устройства;
- **pixelDepth** – содержит глубину пикселя экрана;
- **width** – содержит ширину экрана в пикселях (сравните с availWidth).




**example_6**.

```
<button id="btn">Показать свойства объекта Screen</button><p>
<div id="con"></div>
<script>
    btn.addEventListener("click", ()=>{
        str = `
            availLeft:  ${screen.availLeft} <br>
            availTop: ${screen.availTop} <br>
            availHeight: ${screen.availHeight} <br>
            availWidth: ${screen.availWidth} <br>
            colorDepth: ${screen.colorDepth} <br>
            height: ${screen.height} <br>
            orientation: ${screen.orientation} <br> <!--объект
            ScreenOrientation-->
```

```
            pixelDepth: ${screen.pixelDepth} <br>
            width: ${screen.width}
        `;
        // объект Screen
        console.log(window.screen);
        // объект ScreenOrientation
        console.log(screen.orientation);
        document.getElementById('con').innerHTML = str;
    });
</script>
```

# Диалоговые окна window

Для взаимодействия с пользователем в объекте **window** определен ряд методов, которые позволяют создавать диалоговые окна или взаимодействуют с содержимым окна:

- **alert**() – выводит окно с сообщением;
- **confirm**() – отображает окно с сообщением, в котором пользователь должен подтвердить действие двух кнопок **OK** и **Отмена**;
- **prompt**() – позволяет с помощью диалогового окна запрашивать у пользователя какие-либо данные;
- **print**() – отображает диалоговое окно для вывода страницы на печать.

## alert

Метод **alert**() выводит стандартное окно с сообщением, один из самых востребованных методов объекта **window**.

И если в первом примере мы переопределили метод переменной, то теперь переопределим **стандартный** метод **пользовательской функцией**.

**example_7**.

```
<script>
    // стандартный метод и стандартное действие
    window.alert("Привет мир!");
    // переопределим стандартный метод
    var alert = function(message){
        document.write ("<h1>", message, "</h1>");
    };
    // стандартный метод и нестандартное действие
    window.alert("Привет мир!");
</script>
```

## confirm

Метод **confirm**() отображает окно с сообщением, в котором пользователь должен подтвердить действие двух кнопок **OK** и **Отмена**.

В зависимости от выбора пользователя метод возвращает **true** (если пользователь нажал OK) или **false** (если пользователь нажал кнопку Отмена)

**example_8**.

```
<a href="https://vk.com/pechora_pro"
onclick="test()">Образовательный контент здесь ;)</a>
<script>
    function test() {
        if (! confirm("Оно вам надо?"))
            event.preventDefault();
    }
</script>
```

**example_9**.

```
<h3>Этот контент не для слабонервных ...</h3>
<div style="display:none" id="block">
    <h2>Все, кого ты любил, ... (Чак Паланик).</h2>
</div>
<button id="btn">Все равно показать</button>
<script>
    const btn = document.getElementById("btn");
    btn.addEventListener("click", ()=>{
        if (confirm("Вы уверены?")) {
            document.getElementById('block').style.display =
            'block';
            event.target.style.display = 'none';
        }
    });
</script>
```

## prompt

Метод **prompt**() позволяет с помощью диалогового окна запрашивать у пользователя какие-либо данные. Данный метод возвращает **введенное пользователем** значение.

Например, запросим на просмотр данные группы из подключаемого массива.

**example_10**. **example_10.html**

```
<!-- подключаем массив -->
<script src="teams.js"></script>
```

```
<script>
    // console.log(teams)
    var name = prompt('Какую группу ищем?', 'Aerosmith')
    for (item of teams) {
        if (item[1] == name) {
            console.log('Вы искали:');
            console.log(item);
            break;
        }
    }
</script>
```

**example_10**. **teams.js**

```
let teams =
    [
        [1, "Aerosmith", "aerosmith", "США", "Группа Aerosmith
        ...", 1970, "хард-рок",
        "https://ru.wikipedia.org/wiki/Aerosmith"],
        // ...
    ]
```

## print

Метод **print**() отображает диалоговое сообщение для вывода страницы на печать.

**example_11**.

```
<button id="btn">Печать</button>
```

```
<script>
    const btn = document.getElementById("btn");
    btn.addEventListener("click", ()=>{
        // выводим текущую страницу на печать
        print();
    });
</script>
```

# Поиск на странице

Метод **find**() позволяет найти на странице определенный текст.

> ***Примечание***. Метод **find**() не входит в стандартную комплектацию и не является частью какой-либо спецификации.

Не рекомендуется использовать функцию поиска с использованием метода на реальных сайтах, имеющих выход в глобальную сеть Интернет. Между реализациями может быть большая несовместимость, и поведение метода может измениться в будущем.

**example_12**.

```
<ol>
    <li><span>Зеленая миля</span>:
        <ul>
            <li>режиссёр: Фрэнк Дарабонт</li>
            <li>год: 1999</li>
            <li>страна: США</li>
        </ul>
```

```
    </li>
    <!-- ... -->
</ol>
Что ищем:
<input id="keyWord" type="text" value="Зеленая миля" />
<button id="btn">Поиск</button>
<script>
    const btn = document.getElementById("btn");
    const keyWord = document.getElementById("keyWord");
    btn.addEventListener("click", ()=>{
        // ищем введенное в поле слово
        const result = find(keyWord.value);
            console.log(result);
    });
</script>
```

В некоторых браузерах метод **find**() выделяет (подсвечивает) найденный фрагмент в контенте сайта.

В демонстрационном примере **example_13** рассмотрим **циклический** поиск с использованием метода **find**().

**example_13**.

```
<ol>
    <li><span>Зеленая миля</span>:
        <ul>
            <li>режиссёр: Фрэнк Дарабонт</li>
            <li>год: 1999</li>
            <li>страна: США</li>
```

```
        </ul>
    </li>
    <!-- ... -->
</ol>
Что ищем:
<input id="keyWord" type="text" value="США" />
<button id="btn">Поиск</button>
<script>
    const btn = document.getElementById("btn");
    const keyWord = document.getElementById("keyWord");
    btn.addEventListener("click", ()=>{
        // ищем введенное в поле слово
        let coin = 0;
        while (window.find('США')) {
            coin++;
            res = confirm('Найдено ' + coin + ' совпадение(й). 
            Продолжить поиск?');
            if (!res) break;
        };
        alert ('По вашему запросу найдено ' + coin + ' 
        совпадение(й).')
    });
</script>
```

***Примечание***. Метод **find**() не используется для сбора информации. Это простой **пользовательский поиск** и подсветка (по возможности) некоторой

информации в тексте сайта.

В **самостоятельных работах** к уроку выполним следующие задания:

- Поменять описания стандартных методов alert() и prompt() местами.
- Автоматическая подгрузка и вывод цитат через заданный промежуток времени.
- Отправка формы аутентификации пользователя без использования кнопки submit.

---

**P.S.**

Для отработки и закрепления учебного курса **в расширенной платной версии** предоставляется следующий **раздаточный** материал.

К каждому **уроку** курса:

- Файлы **демонстрационного кода** (example).
- **Задачи** с решениями в контексте рассматриваемых вопросов урока (task).

К каждой **теме** курса:

- **Практические работы**.

К **курсу**:

- **Дополнительный практикум.**